

PostgreSQL Mini Guide

Just enough SQL to build apps and pass interviews, with real outputs to compare against

This guide teaches PostgreSQL with a tiny practice database: **users** who own **todos**. Every query has its **real output** printed underneath, so you can type it yourself and check that you got the same thing.

- Part 1** How PostgreSQL is organised, and setting up the practice database
- Part 2** Reading data: SELECT, WHERE, ORDER BY, LIMIT
- Part 3** Counting and grouping: COUNT, GROUP BY, HAVING
- Part 4** JOINS: combining tables
- Part 5** Changing data, constraints, transactions
- Part 6** Changing tables, indexes, using SQL from Node.js
- Part 7** Exercises (with answers) and interview questions

Part 1: The big picture

Server → databases → tables → rows.

How it's organised

the layers

```
PostgreSQL server  (one program, port 5432, installed once per computer)
├─ todo_app        (a database: your app's data)
│  └─ todos         (a table)
└─ practice_db     (another database: this guide)
   ├── users        (a table: columns id, name, email, created_at)
   └─ todos         (a table: columns id, user_id, text, done, priority)
```

Word	Meaning	MongoDB word
Database	A container for one app's tables	Database
Table	A list of things of one kind, with fixed columns	Collection
Row	One thing (one user, one todo)	Document
Column	One field, with one type, the same in every row	Field
Primary key	The column that uniquely identifies a row (<code>id</code>)	<code>_id</code>
Foreign key	A column pointing at another table's primary key (<code>todos.user_id</code> → <code>users.id</code>)	A stored ObjectId reference

1 Set up the practice database

Create a separate database so you never touch your app's data, then load the practice file. Run these from the `todo-app` folder:

PowerShell

```
psql -U postgres -h localhost -c "CREATE DATABASE practice_db;"
psql -U postgres -h localhost -d practice_db -f guide/practice/practice.sql
```

Then open it and type the queries in this guide:

PowerShell

```
psql -U postgres -h localhost -d practice_db
```

TIP

Made a mess? Run the `-f guide/practice/practice.sql` command again. It deletes and recreates both tables with the starting data. In pgAdmin you can also open the file in the Query Tool and press F5.

2 Read the practice tables

guide/practice/practice.sql (tables)

```
CREATE TABLE users (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        TEXT NOT NULL,
  email       TEXT NOT NULL UNIQUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE todos (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  text        TEXT NOT NULL CHECK (length(text) > 0),
  done        BOOLEAN NOT NULL DEFAULT false,
  priority    INTEGER NOT NULL DEFAULT 2 CHECK (priority BETWEEN 1 AND 3)
);
```

Piece	Meaning
GENERATED ALWAYS AS IDENTITY	Postgres numbers the rows 1, 2, 3... The modern version of <code>SERIAL</code> (your app uses <code>SERIAL</code> ; both work).
PRIMARY KEY	Unique and never empty. Every table should have one.
NOT NULL	A value is required.
UNIQUE	No two rows can have the same value (no duplicate emails).
DEFAULT ...	The value used when you don't give one.
CHECK (...)	A rule every row must pass, e.g. priority must be 1, 2 or 3.
REFERENCES users(id)	Foreign key: <code>user_id</code> must be a real user's id.
ON DELETE CASCADE	When a user is deleted, delete their todos too.

The starting data: Ana has 3 todos, Ben has 2, and **Cy has none** (on purpose, for the JOIN section).

Data types you'll actually use

Type	Holds	Example
------	-------	---------

INTEGER / BIGINT	Whole numbers (BIGINT for huge ones)	42
NUMERIC(10,2)	Exact decimals: use for money	19.99
TEXT	Any string. VARCHAR(n) = with a max length	'Buy milk'
BOOLEAN	true / false (psql prints <code>t</code> / <code>f</code>)	true
DATE	A calendar day	'2026-10-03'
TIMESTAMPZ	A moment in time, with time zone. Prefer it over <code>TIMESTAMP</code>	now()
UUID	A random unique id	gen_random_uuid()
JSONB	Any JSON: Mongo-style data inside Postgres	'{"tags":["home"]}'

IN PLAIN ENGLISH

Strings use **single quotes**: 'Ana'. Double quotes mean a *name* (of a table or column), not text. This trips up almost everyone coming from JavaScript.

Part 2: Reading data

SELECT is 80% of what you'll write.

3 SELECT: pick columns

SQL

```
SELECT id, name, email FROM users;
```

Output

```
id | name | email
---+-----+-----
 1 | Ana  | ana@example.com
 2 | Ben  | ben@example.com
 3 | Cy   | cy@example.com
(3 rows)
```

`SELECT *` means "all columns". In app code, list the columns you need; it's clearer and faster.

4 WHERE + ORDER BY: filter and sort

SQL

```
SELECT text, priority FROM todos WHERE done = false ORDER BY priority, text;
```

Output

```
text | priority
-----+-----
Build the to-do app | 1
Practice TypeScript | 2
Buy milk | 3
(3 rows)
```

`ORDER BY priority, text` sorts by priority first, then by text when priorities tie. Add `DESC` for largest first.

5 More ways to filter

SQL

```
SELECT text FROM todos WHERE text ILIKE '%next%';
```

Output

```
      text
-----
Read Next.js docs
(1 row)
```

SQL

```
SELECT text FROM todos WHERE priority IN (1, 3) AND NOT done;
```

Output

```
      text
-----
Build the to-do app
Buy milk
(2 rows)
```

Operator	Meaning
<code>=</code> , <code><></code> , <code><</code> , <code>>=</code>	Equal, not equal, less, greater-or-equal (one <code>=</code> , not <code>===</code>)
<code>AND</code> , <code>OR</code> , <code>NOT</code>	Combine conditions
<code>IN (1, 3)</code>	Equals any value in the list
<code>BETWEEN 1 AND 3</code>	In a range, ends included
<code>LIKE 'Buy%'</code>	Pattern: <code>%</code> = any characters. <code>ILIKE</code> = ignore upper/lower case
<code>IS NULL</code> , <code>IS NOT NULL</code>	Check for empty values. <code>= NULL</code> never works!

6 LIMIT: only the first few

SQL

```
SELECT id, text FROM todos ORDER BY id DESC LIMIT 2;
```

Output

```
id |      text
---+-----
 5 | Practice TypeScript
 4 | Read Next.js docs
(2 rows)
```

"The 2 newest todos." Always pair `LIMIT` with `ORDER BY`, otherwise "first" means "whatever order the database felt like". For page 2, 3...: `LIMIT 10 OFFSET 10`.

Part 3: Counting and grouping

Turning many rows into a few numbers.

7 COUNT and friends

SQL

```
SELECT COUNT(*) FROM todos;
```

Output

```
count
-----
      5
(1 row)
```

Other **aggregate** functions: `SUM(x)`, `AVG(x)`, `MIN(x)`, `MAX(x)`.

8 GROUP BY: one result per group

SQL

```
SELECT done, COUNT(*) AS how_many FROM todos GROUP BY done ORDER BY done;
```

Output

```
done | how_many
-----+-----
f    |          3
t    |          2
(2 rows)
```

IN PLAIN ENGLISH

GROUP BY sorts the rows into piles (here: `done = false`, `done = true`), then `COUNT(*)` counts each pile. `AS how_many` names the result column.

9 HAVING: filter the groups

SQL

```
SELECT user_id, COUNT(*) AS total FROM todos GROUP BY user_id HAVING COUNT(*) > 2;
```

Output

```
user_id | total
-----+-----
      1 |      3
(1 row)
```

WHERE filters rows *before* grouping; **HAVING** filters groups *after*. "Users with more than 2 todos" needs HAVING, because the count only exists after grouping.

Part 4: JOINS

The reason relational databases exist.

10 JOIN: combine rows from two tables

A todo stores only `user_id`. To show the user's **name** next to each todo, join the two tables where the ids match:

SQL

```
SELECT users.name, todos.text
FROM todos
JOIN users ON users.id = todos.user_id
ORDER BY users.name, todos.id;
```

Output

```
name |      text
-----+-----
Ana   | Learn SQL
Ana   | Build the to-do app
Ana   | Buy milk
Ben   | Read Next.js docs
Ben   | Practice TypeScript
(5 rows)
```

IN PLAIN ENGLISH

Read `JOIN users ON users.id = todos.user_id` as: "for each todo, find the user whose id equals this todo's `user_id`, and glue that user's columns onto the row". Write `table.column` when both tables have a column with the same name (both have `id`).

11 LEFT JOIN: keep rows with no match

Count todos per user. Compare the two kinds of join:

SQL: JOIN (inner)

```
SELECT users.name,
       COUNT(todos.id) AS todo_count
FROM users
JOIN todos
  ON todos.user_id = users.id
GROUP BY users.name
ORDER BY users.name;
```

Output

```
name | todo_count
-----+-----
Ana   |          3
Ben   |          2
(2 rows)
```

SQL: LEFT JOIN

```
SELECT users.name,
       COUNT(todos.id) AS todo_count
FROM users
LEFT JOIN todos
  ON todos.user_id = users.id
GROUP BY users.name
ORDER BY users.name;
```

Output

```
name | todo_count
-----+-----
Ana   |          3
Ben   |          2
Cy    |          0
(3 rows)
```

Join	Keeps	Cy (no todos)
<code>JOIN (= INNER JOIN)</code>	Only rows that match on both sides	Disappears
<code>LEFT JOIN</code>	Every row of the left table (<code>users</code>); missing right side becomes NULL	Kept, with 0

TIP

Use `COUNT(todos.id)`, not `COUNT(*)`, with a `LEFT JOIN`. `COUNT(column)` skips NULLs, so Cy gets 0. `COUNT(*)` counts the row itself and would give Cy 1.

Part 5: Changing data

INSERT, UPDATE, DELETE, and the rules that protect your data.

WATCH OUT

The queries in this part **change** the data, and each one continues from the previous. Run them in order. To start over, reload `practice.sql`.

12 INSERT ... RETURNING

SQL

```
INSERT INTO todos (user_id, text) VALUES (3, 'Say hi') RETURNING id, text, done, priority;
```

Output

id	text	done	priority
6	Say hi	f	2

(1 row)

INSERT 0 1

Columns you leave out get their `DEFAULT` (done = false, priority = 2). `RETURNING` hands back the new row, which is how your app's `createTodo` gets the new `id`.

13 UPDATE

SQL

```
UPDATE todos SET done = true WHERE id = 2 RETURNING id, text, done;
```

Output

id	text	done
2	Build the to-do app	t

(1 row)

UPDATE 1

14 DELETE

SQL

```
DELETE FROM todos WHERE done = true;
```

Output

DELETE 3

WATCH OUT

UPDATE or DELETE without WHERE changes every row. `DELETE FROM todos;` empties the table, with no undo. Write the WHERE first, or test it as a SELECT: `SELECT * FROM todos WHERE done = true;`, then swap `SELECT *` for `DELETE`.

15 Constraints say no

This is the big win over plain JavaScript objects: the database itself refuses bad data, even if your app has a bug. Each of these fails:

SQL: UNIQUE

```
INSERT INTO users (name, email) VALUES ('Ana Two', 'ana@example.com');
```

Output

ERROR: duplicate key value violates unique constraint "users_email_key"
DETAIL: Key (email)=(ana@example.com) already exists.

SQL: FOREIGN KEY

```
INSERT INTO todos (user_id, text) VALUES (99, 'Ghost task');
```

Output

ERROR: insert or update on table "todos" violates foreign key constraint "todos_user_id_fkey"
DETAIL: Key (user_id)=(99) is not present in table "users".

SQL: CHECK

```
INSERT INTO todos (user_id, text, priority) VALUES (1, 'Too urgent', 5);
```

Output

ERROR: new row for relation "todos" violates check constraint "todos_priority_check"
DETAIL: Failing row contains (8, 1, Too urgent, f, 5).

SQL: NOT NULL

```
INSERT INTO todos (user_id) VALUES (1);
```

Output

ERROR: null value in column "text" of relation "todos" violates not-null constraint
DETAIL: Failing row contains (9, 1, null, f, 2).

IN PLAIN ENGLISH

Notice the ids **8** and **9** in the last two errors, although the last successful insert was 6. Every attempt takes the next number, even if it fails. **Ids can have gaps**; never assume they're 1, 2, 3 with nothing missing.

16 ON DELETE CASCADE

SQL

```
DELETE FROM users WHERE email = 'ben@example.com';
```

Output

```
DELETE 1
```

SQL

```
SELECT COUNT(*) FROM todos WHERE user_id = 2;
```

Output

```
count
-----
      0
(1 row)
```

Deleting Ben deleted Ben's todos automatically. Without `ON DELETE CASCADE`, the delete would fail with a foreign key error instead. Both are valid designs.

17 Transactions: all or nothing

SQL

```
BEGIN;
UPDATE todos SET text = 'OOPS';
ROLLBACK;
SELECT id, text FROM todos ORDER BY id;
```

Output

```
BEGIN
UPDATE 2
ROLLBACK
 id |  text
-----+-----
  3 | Buy milk
  6 | Say hi
(2 rows)
```

IN PLAIN ENGLISH

Between `BEGIN` and `COMMIT`, changes are temporary. `ROLLBACK` throws them all away; that "oops, no WHERE" update never happened. The classic example is a bank transfer: take money from A *and* give it to B. Both must happen, or neither.

Part 6: Tables, indexes and Node.js

18 ALTER TABLE: change a table that already has data

SQL

```
ALTER TABLE todos ADD COLUMN due_date DATE;
```

Output

```
ALTER TABLE
```

Existing rows get NULL in the new column. Others: `DROP COLUMN due_date`, `RENAME COLUMN text TO title`, and `DROP TABLE todos`; to delete a whole table.

19 Indexes: make lookups fast

SQL

```
CREATE INDEX todos_user_id_idx ON todos (user_id);
```

Output

```
CREATE INDEX
```

SQL

```
EXPLAIN SELECT * FROM todos WHERE user_id = 1;
```

Output

QUERY PLAN

```
-----  
Seq Scan on todos (cost=0.00..1.02 rows=1 width=49)  
  Filter: (user_id = 1)  
(2 rows)
```

IN PLAIN ENGLISH

An **index** is like the index at the back of a book: instead of reading every page (**Seq Scan** = check every row), jump straight to the right ones. `EXPLAIN` shows Postgres's plan.

Here it *still* chose a Seq Scan, because the table has 2 rows and reading them all is faster than using the index. With thousands of rows, the plan switches to an **Index Scan**. Indexes speed up reads but slow down writes slightly and take space, so add them on columns you search or join by (like `user_id`), not everywhere. Primary keys and UNIQUE columns get one automatically.

20 Inspect a table: \d

psql

```
\d todos
```

Output

```

Table "public.todos"
  Column | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
id       | integer       |           | not null | generated always as identity
user_id  | integer       |           | not null |
text     | text          |           | not null |
done     | boolean       |           | not null | false
priority | integer       |           | not null | 2
due_date | date          |           |          |

Indexes:
    "todos_pkey" PRIMARY KEY, btree (id)
    "todos_user_id_idx" btree (user_id)

Check constraints:
    "todos_priority_check" CHECK (priority >= 1 AND priority <= 3)
    "todos_text_check" CHECK (length(text) > 0)

Foreign-key constraints:
    "todos_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

```

Everything you defined, and the names Postgres gave your constraints. Those names are what you saw in the error messages.

21 Using SQL from Node.js (the pg library)

You write	You get back
<code>await pool.query("SELECT ...")</code>	<code>result.rows</code> : an array of plain objects, like <code>[{ id: 1, text: "Buy milk", done: false }]</code>
<code>await pool.query("UPDATE ...")</code>	<code>result.rowCount</code> : how many rows changed
<code>pool.query("... WHERE id = \$1", [id])</code>	Values go in the array, never inside the SQL string

WATCH OUT

SQL injection. Never build SQL by gluing strings: `"... WHERE email = '" + email + "'"`. A user could type `' OR '1'='1` and change your query. With `$1`, the value is sent separately and can never become SQL. For LIKE searches, add the `%` to the value: `pool.query("... WHERE text ILIKE $1", ["%" + q + "%"])`.

A **transaction** needs one connection for every statement, so borrow one from the pool:

lib/transfer-example.ts (type-checked)

```
import { pool } from "@lib/db";

// Move every unfinished todo from one user to another: all or nothing.
export async function reassignTodos(fromUserId: number, toUserId: number): Promise<number> {
  const client = await pool.connect(); // borrow ONE connection for the whole transaction
  try {
    await client.query("BEGIN");
    const result = await client.query(
      "UPDATE todos SET user_id = $1 WHERE user_id = $2 AND NOT done",
      [toUserId, fromUserId]
    );
    await client.query("COMMIT");
    return result.rowCount ?? 0;
  } catch (err) {
    await client.query("ROLLBACK"); // undo everything since BEGIN
    throw err;
  } finally {
    client.release(); // always give the connection back
  }
}
```

Part 7: Practice

Reload practice.sql first, so you start from the original data.

Exercises

Try each one yourself before looking at the answers on the next page.

1. List the names of users who have at least one **unfinished** todo (each name once).
2. How many todos are there per priority, priority 1 first?
3. Which users have **no** todos at all?
4. Show every user with their number of **unfinished** todos, **including** users with zero.
5. Mark all of Ben's todos as done, finding Ben by his email (not by typing his id).

Answers

1. Users with unfinished todos

SQL

```
SELECT DISTINCT users.name
FROM users
JOIN todos ON todos.user_id = users.id
WHERE NOT todos.done
ORDER BY users.name;
```

Output

```
name
-----
Ana
Ben
(2 rows)
```

`DISTINCT` removes duplicates; Ana has two unfinished todos but appears once.

2. Todos per priority

SQL

```
SELECT priority, COUNT(*) AS how_many
FROM todos
GROUP BY priority
ORDER BY priority;
```

Output

```
priority | how_many
-----+-----
1 | 2
2 | 2
3 | 1
(3 rows)
```

3. Users with no todos

SQL

```
SELECT users.name
FROM users
LEFT JOIN todos ON todos.user_id = users.id
WHERE todos.id IS NULL;
```

Output

```
name
-----
Cy
(1 row)
```

`LEFT JOIN` keeps Cy with NULLs on the todo side; `WHERE todos.id IS NULL` keeps only those.

4. Unfinished todos per user, including zero (the classic trap)

SQL: correct (condition in ON)

```
SELECT users.name,  
       COUNT(todos.id) AS open_todos  
FROM users  
LEFT JOIN todos  
  ON todos.user_id = users.id  
  AND NOT todos.done  
GROUP BY users.name  
ORDER BY users.name;
```

Output

name	open_todos
Ana	2
Ben	1
Cy	0

(3 rows)

SQL: wrong (condition in WHERE)

```
SELECT users.name,  
       COUNT(todos.id) AS open_todos  
FROM users  
LEFT JOIN todos  
  ON todos.user_id = users.id  
WHERE NOT todos.done  
GROUP BY users.name  
ORDER BY users.name;
```

Output

name	open_todos
Ana	2
Ben	1

(2 rows)

IN PLAIN ENGLISH

Both look almost the same, but in the wrong version **Cy disappears**. The LEFT JOIN gives Cy a row with NULLs, then `WHERE NOT todos.done` runs, and NULL isn't "not done", so the row is thrown away. Putting the condition in `ON` filters the todos *while joining*, so Cy survives with 0. This exact bug is a favourite interview question.

5. Update using a subquery

SQL

```
UPDATE todos SET done = true  
WHERE user_id = (SELECT id FROM users WHERE email = 'ben@example.com')  
RETURNING text, done;
```

Output

text	done
Read Next.js docs	t
Practice TypeScript	t

(2 rows)

UPDATE 2

The query in brackets runs first and returns Ben's id. Real apps rarely know ids by heart, so this pattern is common.

Interview questions, short answers

Question	Short answer
SQL vs NoSQL?	SQL: fixed tables, relationships, JOINS, strong rules (users, orders, payments). NoSQL (e.g. MongoDB): flexible documents, nested data, easy to change shape. Postgres can also store JSON in JSONB columns.

Primary key vs foreign key?	A primary key uniquely identifies a row in its own table. A foreign key is a column that points to another table's primary key and must match a real row.
INNER vs LEFT JOIN?	INNER keeps only matching rows. LEFT keeps every row of the left table and fills missing matches with NULL.
WHERE vs HAVING?	WHERE filters rows before grouping; HAVING filters groups after GROUP BY.
What is an index? Downsides?	A lookup structure that finds rows without scanning the whole table. It costs disk space and slows down inserts and updates a little.
What is a transaction / ACID?	A group of statements that succeed or fail together. ACID: Atomic (all or nothing), Consistent (rules hold), Isolated (transactions don't see each other's half-done work), Durable (committed data survives a crash).
What is normalization?	Storing each fact once and linking with keys, e.g. the user's name lives in <code>users</code> only, and <code>todos</code> stores <code>user_id</code> . That avoids copies getting out of sync.
How do you prevent SQL injection?	Parameterized queries (<code>\$1</code> , <code>\$2</code>), never string concatenation.
What is the N+1 problem?	Loading 100 todos, then running 1 extra query per todo for its user (101 queries). Fix it with one JOIN.

INTERVIEW ANGLE

When asked to write a query, say your plan out loud first: *"I need users and their todos, so I'll join on user_id; I want users with zero too, so LEFT JOIN; then group by user and count."* Interviewers grade the reasoning as much as the final SQL.